

KONYA FOOD AND AGRICULTURE UNIVERSITY

Faculty of Engineering of Computer Engineering



Hospital Management System

Microservices Architecture

A Polyglot Microservices Project 7

Prepared by

Betül Korkmaz

Student No: 2120100200113

Fatma Zehra Bestil

Student No: 212010020058

May 2026

TABLE OF CONTENTS

1. Executive Summary & Project Overview	4
1.1 Project Description	4
1.2 Goals & Motivation	4
1.3 Five Microservices at a Glance.....	4
1.4 Technology Overview.....	5
2. System Architecture.....	6
2.1 Microservices Architecture Pattern	6
2.2 Docker Compose Orchestration	6
2.3 Service Communication Map.....	7
2.4 Network Architecture	7
3. Patient Service — Spring Boot.....	8
3.1 Framework Overview.....	8
3.2 REST API Endpoints	8
3.3 Data Model — Patient	9
3.4 Key Implementation Details.....	9
3.5 LogClient Component	9
4. Appointment Service — Spark Java	10
4.1 Framework Overview.....	10
4.2 REST API Endpoints	10
4.3 Data Model — Appointment	11
4.4 Key Implementation Details.....	11
4.5 Spark Java Route Pattern	12
5. Doctor Service — Dropwizard.....	12
5.1 Framework Overview.....	12
5.2 REST API Endpoints	12
5.3 Data Model — Doctor	13
5.4 Preloaded Sample Data	13
5.5 CORS & Build.....	14
6. Activity Log Service — Eclipse Vert.x	14
6.1 Framework Overview.....	14
6.2 REST API Endpoints	14
6.3 Log Entry Data Model	15
6.4 Verticle Architecture	15
6.5 Why Vert.x for Logging?.....	15
7. Frontend — React 18 + Vite.....	16
7.1 Technology Stack	16

7.2 Application Sections	16
7.3 Sidebar Navigation.....	16
7.4 UI Design Highlights.....	17
7.5 Docker Build.....	17
8. Docker & Deployment	18
8.1 Multi-Stage Docker Builds.....	18
8.2 Port Mappings	18
8.3 Startup Order (depends_on)	18
8.4 Environment Variables.....	19
8.5 Deployment Command.....	19
9. Inter-Service Communication & Logging	20
9.1 Logging Architecture.....	20
9.2 The sendLog() Pattern.....	20
9.3 Log Events by Service.....	20
10. CORS Configuration	22
10.1 Why CORS Is Required	22
10.2 CORS Implementation Per Framework	22
10.3 Spring Boot CORS (WebMvcConfigurer)	22
10.4 Vert.x CORS (Manual Headers)	23
11. Data Models & API Reference.....	24
11.1 Complete REST API Reference	24
11.2 HTTP Status Codes Used	25
12. Testing & Validation	26
12.1 Testing Approach	26
12.2 Health Check Endpoints.....	26
12.3 Sample curl Commands	26
12.4 Verifying Inter-Service Logging	27
12.5 Common Issues & Solutions	27
13. Technologies Summary & Conclusion.....	28
13.1 Complete Technology Stack.....	28
13.2 Key Design Decisions	29
13.3 Conclusion	29
14. References & Appendix	30
14.1 References	30
14.2 Appendix A — docker-compose.yml	31
14.3 Appendix B — Project Directory Structure	32

1. Executive Summary & Project Overview

1.1 Project Description

The Hospital Management System (HMS) is a full-stack, polyglot microservices application designed to simulate a real-world Turkish hospital information system — known as **HBYS (Hastane Bilgi Yönetim Sistemi)**. The project demonstrates how modern software engineering principles — including service decomposition, containerization, reactive programming, and asynchronous interservice communication — can be applied in a healthcare context.

The system is composed of five independent, loosely-coupled services, each built with a different technology stack, showcasing the concept of **polyglot microservices** where the right tool is chosen for each specific task.

1.2 Goals & Motivation

- Demonstrate microservices architecture with multiple Java frameworks
- Implement a realistic HBYS-inspired user interface
- Show containerization with Docker and Docker Compose
- Explore reactive programming with Eclipse Vert.x
- Practice asynchronous inter-service communication
- Compare different REST API frameworks side by side

1.3 Five Microservices at a Glance

Service	Framework	Port	Responsibility
Patient Service	Spring Boot 3.2.0	8081	Patient CRUD, JPA/H2 persistence
Appointment Service	Spark Java 2.9.4	8082	Appointment scheduling, inmemory
Doctor Service	Dropwizard 2.1.6	8083	Doctor management, JAX-RS REST
Activity Log Service	Vert.x 4.5.1	8085	Centralized async logging hub
Frontend	React 18 + Vite 5	3000	HBYS-style single-page application

1.4 Technology Overview

All backend services are written in Java (versions 11 and 17), packaged as self-contained fat JARs, and containerized with multi-stage Docker builds. The frontend is a React 18 SPA built with Vite and served via Nginx. The entire stack is orchestrated with a single `docker-compose.yml` file and starts with one command

2. System Architecture

2.1 Microservices Architecture Pattern

The project follows the **microservices architectural style**, where each business capability is encapsulated in its own independently deployable service. Services communicate over HTTP REST APIs rather than sharing a database, ensuring loose coupling and high cohesion.

Key architectural characteristics:

- Each service owns its data store (or in-memory state)
- Services communicate via synchronous HTTP REST (for business operations)
- Activity logging uses asynchronous fire-and-forget HTTP POST calls
- No shared libraries between services (polyglot principle)
- Each service can be built, deployed, and scaled independently

2.2 Docker Compose Orchestration

Docker Compose orchestrates all five containers within a shared bridge network named `hospitalnetwork`. Service startup order is enforced via `depends_on` directives, ensuring the Activity Log Service starts before any business service attempts to connect.

```
docker-compose.yml structure:

activity-log-service (starts first - no dependencies)
├─ patient-service   (depends_on: activity-log-service)
├─ doctor-service    (depends_on: activity-log-service)
└─ appointment-service (depends_on: activity-log-service, patientservice)
    └─ frontend       (depends_on: all four services)
```

2.3 Service Communication Map

Service	Technology	Port(s)	Dependencies
patient-service	Spring Boot 3.2.0, Java 17, H2 DB	8081	activity-log-service
appointment-service	Spark Java 2.9.4, Java 11, In-memory	8082	activity-log-service, patientservice
doctor-service	Dropwizard 2.1.6, Java 11, In-memory	8083/8084	activity-log-service
activity-log-service	Vert.x 4.5.1, Java 11, In-memory	8085	none (root service)
Service	Technology	Port(s)	Dependencies
frontend	React 18, Vite 5, Nginx	3000	all four services

2.4 Network Architecture

All containers join the `hospital-network` Docker bridge network. This means services can address each other by container name (e.g., `http://activity-log-service:8085`) without needing IP addresses. Only the ports listed in the `ports` section are exposed to the host machine.

Design Note: The environment variable `LOG_SERVICE_URL` is injected into each Java service at container startup. This allows the log URL to be overridden without rebuilding the Docker image — a 12-factor app best practice.

3. Patient Service — Spring Boot

3.1 Framework Overview

The Patient Service is built with **Spring Boot 3.2.0** running on **Java 17**. It represents the most featurerich service in the project, leveraging Spring's auto-configuration, dependency injection, and Spring Data JPA for database access. Data is stored in an embedded **H2 in-memory database** for simplicity.

3.2 REST API Endpoints

Method	Endpoint	Description
GET	/api/patients	Retrieve all patients
GET	/api/patients/{id}	Retrieve patient by ID
GET	/api/patients/tc/{tcNo}	Retrieve patient by TC identity number
POST	/api/patients	Create a new patient record
PUT	/api/patients/{id}	Update an existing patient
DELETE	/api/patients/{id}	Delete a patient record

3.3 Data Model — Patient

Field	Type	Description
id	Long	Auto-generated primary key
name	String	Patient first name
surname	String	Patient last name
tcNo	String	Turkish national identity number (11 digits)
phone	String	Contact phone number
email	String	Email address
age	Integer	Patient age
bloodType	String	Blood type (A+, B-, O+, AB+, etc.)
createdAt	LocalDate	Registration timestamp

3.4 Key Implementation Details

- CORS: WebMvcConfigurer bean with allowedOrigins("*") and all HTTP methods
- Async Logging: LogClient @Component injects into PatientController, calls sendAsync()
- Log events: create (INFO), update (INFO), delete (WARN)
- Build: spring-boot-maven-plugin produces executable fat JAR
- Docker: maven:3.9.5-eclipse-temurin-17 build stage, eclipse-temurin:17-jre runtime

3.5 LogClient Component

```
@Component
public class LogClient {    private
static final String LOG_URL =
    System.getenv().getOrDefault("LOG_SERVICE_URL",
"http://activity-log-service:8085") + "/logs";

    public void log(String message, String level) {
        // Non-blocking — service remains responsive even if log service is down
        httpClient.sendAsync(request, HttpResponse.BodyHandlers.ofString());    }
}
```

4. Appointment Service — Spark Java

4.1 Framework Overview

The Appointment Service uses **Spark Java 2.9.4**, a lightweight micro-framework for Java that embeds a Jetty server and provides an expressive, concise DSL for defining REST routes. This deliberately contrasts with Spring Boot to demonstrate that different problems can be solved with different tools in a microservices architecture.

4.2 REST API Endpoints

Method	Endpoint	Description
GET	/health	Health check endpoint
GET	/api/appointments	List all appointments
GET	/api/appointments/{id}	Get appointment by ID
GET	/api/appointments/patient/{patientId}	Appointments for a patient
GET	/api/appointments/doctor/{doctorId}	Appointments for a doctor
POST	/api/appointments	Create new appointment
PUT	/api/appointments/{id}	Update appointment
DELETE	/api/appointments/{id}	Delete appointment

4.3 Data Model — Appointment

Field	Type	Description
id	long	Auto-generated ID (AtomicLong)
patientId	long	Reference to patient
doctorId	long	Reference to doctor
appointmentDate	String	Date and time (yyyy-MM-dd HH:mm)
description	String	Appointment notes / reason
status	String	BEKLIYOR / TAMAMLANDI / IPTAL
createdAt	String	Creation timestamp

4.4 Key Implementation Details

- All state stored in a static `HashMap<Long, Appointment>`
- IDs generated by `AtomicLong` (thread-safe counter)
- JSON serialization with `Gson 2.10.1` (`GsonBuilder` with pretty printing)
- CORS handled in `before()` filter: sets headers on every response
- Build: `maven-assembly-plugin` creates fat JAR with all dependencies
- Status values use Turkish terminology for realism: `BEKLIYOR` (waiting), `TAMAMLANDI` (completed), `IPTAL` (cancelled)

4.5 Spark Java Route Pattern

```
// Spark Java DSL – entire service in one class port(8082);

before((req, res) -> {
    res.header("Access-Control-Allow-Origin", "*");
    res.header("Access-Control-Allow-Methods", "GET, POST, PUT, DELETE, OPTIONS");
});

get("/api/appointments", (req, res) -> {
    return gson.toJson(new ArrayList<>(appointments.values())); });
```

5. Doctor Service — Dropwizard

5.1 Framework Overview

The Doctor Service is built with **Dropwizard 2.1.6**, a Java framework that bundles Jersey (JAX-RS), Jackson, Jetty, Metrics, and other production-ready libraries into a cohesive stack. Dropwizard is designed for building high-performance RESTful web services with minimal configuration overhead.

5.2 REST API Endpoints

Method	Endpoint	Description
GET	/api/doctors	List all doctors
GET	/api/doctors/{id}	Get doctor by ID
GET	/api/doctors/specialty/{spec}	Filter doctors by specialty
GET	/api/doctors/available	List only available doctors
POST	/api/doctors	Add a new doctor
PUT	/api/doctors/{id}	Update doctor details
DELETE	/api/doctors/{id}	Remove a doctor

5.3 Data Model — Doctor

Field	Type	Description
id	Long	Auto-generated ID (AtomicLong from 5)
name	String	Doctor first name
surname	String	Doctor last name
specialty	String	Medical specialty (e.g., Kardiyoloji)
phone	String	Contact phone number
email	String	Email address
available	boolean	Whether the doctor is currently available

5.4 Preloaded Sample Data

ID	Name	Specialty	Available
1	Ahmet Yilmaz	Kardiyoloji (Cardiology)	Yes
2	Ayşe Demir	Noroloji (Neurology)	Yes
3	Mehmet Kaya	Ortopedi (Orthopedics)	No
4	Fatma Celik	Dahiliye (Internal Medicine)	Yes

5.5 CORS & Build

- CORS: CrossOriginFilter servlet filter registered in DoctorApplication.configure()
- CrossOriginFilter.ALLOWED_ORIGINS_PARAM = "*"
- Build: maven-shade-plugin creates uber-jar
- Exposes two ports: 8083 (application), 8084 (Dropwizard admin console)

```
// Dropwizard CORS registration
FilterRegistration.Dynamic
cors =
    environment.servlets().addFilter("CORS", CrossOriginFilter.class);
cors.setInitParameter(CrossOriginFilter.ALLOWED_ORIGINS_PARAM, "*");
cors.addMappingForUrlPatterns(
    EnumSet.allOf(DispatcherType.class), true, "/*");
```

6. Activity Log Service — Eclipse Vert.x

6.1 Framework Overview

The Activity Log Service is built with **Eclipse Vert.x 4.5.1**, a toolkit for building reactive applications on the JVM. Unlike the other services which use traditional blocking I/O, Vert.x uses an **event-loop model** (similar to Node.js) where all I/O is non-blocking and handlers are executed on a small pool of event-loop threads.

This makes Vert.x ideal for a high-throughput logging service: it can handle thousands of concurrent log POST requests without creating one thread per request, making it extremely memory-efficient.

6.2 REST API Endpoints

Method	Endpoint	Description
GET	/health	Health check — returns service status and log count
GET	/logs	Retrieve all log entries, newest first
POST	/logs	Create a new log entry
DELETE	/logs	Clear all log entries

6.3 Log Entry Data Model

Field	Type	Description
id	Long	Auto-incremented log ID (AtomicLong)
service	String	Name of the originating service
message	String	Human-readable log message
level	String	Severity: INFO WARN ERROR DEBUG
time	String	Formatted timestamp (yyyy-MM-dd HH:mm:ss)

6.4 Verticle Architecture

The service is structured as a single `ActivityLogVerticle` class extending `AbstractVerticle`. The `start(Promise<Void>)` method configures the HTTP router and starts the server. The router handles CORS manually by setting response headers before delegating to route handlers.

```
// Vert.x reactive server — non-blocking event loop
public class ActivityLogVerticle extends AbstractVerticle {
    private static final List<JsonObject> logs = new ArrayList<>();
    private static final AtomicLong idGen = new AtomicLong(1);

    @Override
    public void start(Promise<Void> startPromise) {
        Router router = Router.router(vertx);
        // CORS + BodyHandler setup
        // Route registration: GET/POST/DELETE /logs
        vertx.createHttpServer().requestHandler(router).listen(8085, ...);
    }
}
```

6.5 Why Vert.x for Logging?

- Handles burst log traffic without thread exhaustion
- Low memory footprint — no thread-per-request model
- Fire-and-forget compatible: senders use `sendAsync()` and don't wait
- Simple JSON API makes it easy to integrate from any language

7. Frontend — React 18 + Vite

7.1 Technology Stack

The frontend is a **React 18** single-page application built with **Vite 5** as the build tool. In production (Docker), it is served as static assets via **Nginx** on port 3000. The UI design is inspired by Turkish HBYS systems — particularly **Probel** — to give it a realistic, professional hospital information system aesthetic.

7.2 Application Sections

Section (TR)	Section (EN)	Features
Ana Ekran	Dashboard	Stats cards, recent patients, appointments, doctor status, system info
Hasta Kayıt	Patient Management	CRUD, protocol numbers (P-000001), blood type, TC No, gender
Poliklinik Randevusu	Appointments	Calendar date picker, hourly time select (08:00–18:00), status management
Hekim Listesi	Doctor Management	Specialty filter, availability toggle, Uzm. Dr. prefix
Aktivite Logları	Activity Logs	Real-time auto-refresh (3s), filter by service/level, CANLI indicator

7.3 Sidebar Navigation

The application uses a fixed dark-navy sidebar (`--sidebar-bg: #0d2137`) with grouped navigation sections:

- Genel: Ana Ekran (Dashboard)
- Hasta İşlemleri: Hasta Kayıt, Poliklinik Randevusu
- Personel: Hekim Listesi
- Sistem: Aktivite Logları

7.4 UI Design Highlights

- Real-time digital clock in the top navigation bar
- Color-coded log level badges: INFO (blue), WARN (yellow), ERROR (red), DEBUG (gray)
- Color-coded service badges: Patient (green), Doctor (blue), Appointment (teal)
- MediSoft HBYS branding with Merkez Hastane institution badge
- Responsive modal dialogs for create/edit operations
- Protocol number format: P-000001 for patients
- LIVE indicator (● CANLI) pulsing when auto-refresh is active

7.5 Docker Build

```
# Multi-stage Dockerfile FROM
node:20-alpine AS build
WORKDIR /app
COPY package*.json .
RUN npm install COPY
...
RUN npm run build          # Vite produces /app/dist

FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html EXPOSE
3000
```

8. Docker & Deployment

8.1 Multi-Stage Docker Builds

Every service uses a **multi-stage Dockerfile**. The first stage uses a Maven JDK image to compile the Java source and produce a fat JAR. The second stage copies only the JAR into a slim JRE image, dramatically reducing the final image size.

```
# Generic Java service pattern
FROM maven:3.9.5-eclipse-temurin-11 AS build
WORKDIR /app COPY pom.xml .
COPY src ./src
RUN mvn clean package -DskipTests

FROM eclipse-temurin:11-jre
COPY --from=build /app/target/service-1.0.0.jar app.jar
EXPOSE 8085
ENTRYPOINT ["java", "-jar", "app.jar"]
```

8.2 Port Mappings

Container Name	Service	Host Port	Container Port
activity-log-service	Vert.x Log Hub	8085	8085
patient-service	Spring Boot	8081	8081
appointment-service	Spark Java	8082	8082
doctor-service	Dropwizard	8083, 8084	8083, 8084
frontend	React/Nginx	3000	3000

8.3 Startup Order (depends_on)

The `depends_on` directive in `docker-compose.yml` ensures correct startup sequencing:

- activity-log-service starts first (no dependencies)
- patient-service and doctor-service wait for activity-log-service
- appointment-service waits for both activity-log-service and patient-service
- frontend waits for all four backend services

8.4 Environment Variables

Variable	Value (Docker)	Purpose
LOG_SERVICE_URL	http://activity-log-service:8085	Log service URL — uses container name for DNS

8.5 Deployment Command

```
# Start all 5 services with a single commandcd
/path/to/mikroservis
docker-compose up --build

# Run in background
docker-compose up --build -d

# Stop all services docker-compose
down
```

9. Inter-Service Communication & Logging

9.1 Logging Architecture

All three business services (Patient, Appointment, Doctor) send structured log entries to the Activity Log Service via asynchronous HTTP POST requests. The key design principle is **fire-and-forget**: the log call is non-blocking and a failure in the log service never affects the main service's response.

9.2 The `sendLog()` Pattern

Each service implements a private static `sendLog()` method with the same structure:

```
private static void sendLog(String message, String level) {
    try {
        String body = String.format(
            "{\"service\":\"Patient
Service\",\"message\":\"%s\",\"level\":\"%s\"}",
            message.replace("\"", ""), level);

        HttpRequest req = HttpRequest.newBuilder()
            .uri(URI.create(LOG_URL))
            .header("Content-Type", "application/json")
            .POST(HttpRequest.BodyPublishers.ofString(body))
            .build();

        // Non-blocking — does NOT wait for response
        httpClient.sendAsync(req, HttpResponse.BodyHandlers.ofString());

    } catch (Exception ignored) {} // Log failure never crashes main service
}
```

9.3 Log Events by Service

Service	Event	Level
Patient Service	Service startup	INFO
Patient Service	New patient created	INFO
Patient Service	Patient record updated	INFO
Patient Service	Patient record deleted	WARN
Appointment Service	Service startup	INFO

Appointment Service	New appointment created	INFO
Appointment Service	Appointment updated	INFO
Service	Event	Level
Appointment Service	Appointment deleted	WARN
Doctor Service	New doctor added	INFO
Doctor Service	Doctor record updated	INFO
Doctor Service	Doctor record deleted	WARN
Activity Log Service	Service startup	INFO

10. CORS Configuration

10.1 Why CORS Is Required

The frontend runs at `http://localhost:3000` and calls APIs at ports 8081–8085. Modern browsers enforce the **Same-Origin Policy**: a page may only make XMLHttpRequest or Fetch calls to the same origin (protocol + host + port). Since the ports differ, all API calls would be blocked without CrossOrigin Resource Sharing (CORS) headers.

Each backend service must respond with `Access-Control-Allow-Origin: *` (or a specific origin) and handle `OPTIONS` preflight requests from the browser.

10.2 CORS Implementation Per Framework

Service	Framework	CORS Mechanism
patient-service	Spring Boot	WebMvcConfigurer bean — addCorsMappings() registry
appointment-service	Spark Java	before() filter — sets headers on every response
doctor-service	Dropwizard	CrossOriginFilter registered as servlet filter
activity-log-service	Vert.x	Manual headers in route handler, OPTIONS returns 200

10.3 Spring Boot CORS (WebMvcConfigurer)

```
@Bean
public WebMvcConfigurer corsConfigurer() {
    return new WebMvcConfigurer() {
        @Override
        public void
        addCorsMappings(CorsRegistry registry) {
            registry.addMapping("/**")
                .allowedOrigins("*")
                .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS")
                .allowedHeaders("*");
        }
    };
}
```

10.4 Vert.x CORS (Manual Headers)

```
router.route().handler(ctx -> {  
  ctx.response()  
  
    .putHeader("Access-Control-Allow-Origin", "*")  
    .putHeader("Access-Control-Allow-Methods", "GET, POST, DELETE, OPTIONS")  
    .putHeader("Access-Control-Allow-Headers", "Content-Type");  
  if (ctx.request().method() == HttpMethod.OPTIONS) {  
    ctx.response().setStatusCode(200).end();  
  } else {  
    ctx.next();  
  }  
});
```

11. Data Models & API Reference

11.1 Complete REST API Reference

Method	Service	Endpoint	Status Codes
GET	Patient	/api/patients	200
POST	Patient	/api/patients	201, 400
PUT	Patient	/api/patients/{id}	200, 404
DELETE	Patient	/api/patients/{id}	200, 404
GET	Appointment	/api/appointments	200
POST	Appointment	/api/appointments	201
PUT	Appointment	/api/appointments/{id}	200, 404
DELETE	Appointment	/api/appointments/{id}	200, 404
GET	Doctor	/api/doctors	200
GET	Doctor	/api/doctors/available	200
POST	Doctor	/api/doctors	201
PUT	Doctor	/api/doctors/{id}	200, 404
DELETE	Doctor	/api/doctors/{id}	200, 404
GET	Logs	/logs	200
POST	Logs	/logs	201
DELETE	Logs	/logs	200

11.2 HTTP Status Codes Used

Code	Meaning	When Returned
200	OK	Successful GET, PUT, DELETE
201	Created	Successful POST (resource created)
Code	Meaning	When Returned
400	Bad Request	Invalid request body or parameters
404	Not Found	Resource with given ID does not exist

12. Testing & Validation

12.1 Testing Approach

The project uses manual integration testing with live Docker containers rather than unit tests with mocks. This approach was chosen to validate actual inter-service communication, network configuration, and end-to-end data flow under realistic conditions.

12.2 Health Check Endpoints

Each service exposes a `/health` endpoint that returns service status and key metrics:

URL	Expected Response
<code>http://localhost:8081/actuator/health</code>	<code>{"status":"UP"}</code>
<code>http://localhost:8082/health</code>	<code>{"status":"UP","service":"appointmentservice","totalAppointments":2}</code>
<code>http://localhost:8083/healthcheck</code>	Dropwizard health check (JSON)
<code>http://localhost:8085/health</code>	<code>{"status":"UP","framework":"Vert.x 4.5.1","totalLogs":5}</code>

12.3 Sample curl Commands

```
# List all patients
curl http://localhost:8081/api/patients

# Create a new patient
curl -X POST http://localhost:8081/api/patients \
  -H 'Content-Type: application/json' \
  -d '{"name":"Ali","surname":"Veli","tcNo":"12345678901","bloodType":"A+"}'

# Check appointment service health
curl http://localhost:8082/health

# List available doctors
curl http://localhost:8083/api/doctors/available

# View all logs
curl http://localhost:8085/logs

# Clear all logs
curl -X DELETE http://localhost:8085/logs
```

12.4 Verifying Inter-Service Logging

- Open the frontend at <http://localhost:3000>
- Navigate to Hasta Kayıt (Patient Management)
- Create a new patient via the modal form
- Navigate to Aktivite Logları (Activity Logs)
- Verify the new log entry appears: "Yeni hasta kaydı oluşturuldu: [name]" at INFO level
- Delete the patient and verify a WARN level log appears

12.5 Common Issues & Solutions

Issue	Cause	Solution
CORS error in browser	Missing CORS headers	Verify the service is running and CORS config is applied
Port already in use	Previous container still running	Run: <code>docker-compose down</code>
Log service not receiving	Container name DNS not resolved	Ensure services are on hospitalnetwork
Frontend blank page	API URLs wrong	Check VITE_ env vars or API constants in components

13. Technologies Summary & Conclusion

13.1 Complete Technology Stack

Technology	Version	Role in Project
Spring Boot	3.2.0	Patient Service — REST API, JPA, H2 DB, DI
Spark Java	2.9.4	Appointment Service — lightweight embedded HTTP
Dropwizard	2.1.6	Doctor Service — JAX-RS, Jersey, production-ready
Eclipse Vert.x	4.5.1	Activity Log Service — reactive, event-loop HTTP
React	18	Frontend — component-based SPA
Vite	5	Frontend build tool — fast HMR and bundling
Nginx	latest	Static file server for production frontend
Docker	latest	Service containerization
Docker Compose	latest	Multi-container orchestration
Java	17	Patient Service JVM (LTS)
Java	11	Appointment, Doctor, Log Service JVM (LTS)
H2 Database	embedded	Patient data persistence (in-memory)
Gson	2.10.1	JSON serialization in Appointment Service
Jackson	bundled	JSON in Spring Boot and Dropwizard

13.2 Key Design Decisions

- Intentionally polyglot: Each service uses a different Java framework to demonstrate comparative strengths
- Fire-and-forget logging: Async sendAsync() prevents log failures from cascading
- In-memory state: Keeps the project focused on architecture rather than database setup
- Turkish UI terminology: Mirrors real HBYS systems used in Turkish hospitals
- Multi-stage Docker builds: Separates build environment from runtime, minimizing image size

13.3 Conclusion

The Hospital Management System successfully demonstrates a modern microservices architecture using five different technology stacks. The project shows how services can be independently developed, containerized, and orchestrated while still communicating effectively through welldefined HTTP APIs.

The integration of **Eclipse Vert.x** as a reactive logging hub, combined with Spring Boot, Spark Java, and Dropwizard business services, illustrates that polyglot architectures are not only feasible but practical. The React frontend ties all services together in a realistic, professional HBYS-inspired interface.

This project serves as a comprehensive educational reference for microservices design patterns, Docker deployment strategies, CORS configuration across frameworks, and reactive programming with Vert.x.

14. References & Appendix

14.1 References

Framework / Tool	Official Documentation
Spring Boot 3.2	https://docs.spring.io/spring-boot/docs/3.2.0/reference/html/
Spark Java 2.9.4	http://sparkjava.com/documentation
Dropwizard 2.1	https://www.dropwizard.io/en/stable/
Eclipse Vert.x 4.5	https://vertx.io/docs/
React 18	https://react.dev/
Vite 5	https://vitejs.dev/guide/
Docker Compose	https://docs.docker.com/compose/
Gson 2.10.1	https://github.com/google/gson

14.2 Appendix A — docker-compose.yml

```

version: '3.8'
services:
  activity-log-service:
    build: ./activity-log-service
    container_name: activity-log-service
    ports: ["8085:8085"]
    networks: [hospital-network]
    restart: unless-stopped

  patient-service:
    build: ./patient-service
    ports: ["8081:8081"]
    depends_on: [activity-log-service]
    environment:
      - LOG_SERVICE_URL=http://activity-log-service:8085
    networks: [hospital-network]

  appointment-service:
    build: ./appointment-service
    ports: ["8082:8082"]
    depends_on: [activity-log-service, patient-service]
    environment:
      - LOG_SERVICE_URL=http://activity-log-service:8085
    networks: [hospital-network]

  doctor-service:
    build: ./doctor-service
    ports: ["8083:8083", "8084:8084"]
    depends_on: [activity-log-service]
    environment:
      - LOG_SERVICE_URL=http://activity-log-service:8085
    networks: [hospital-network]

  frontend:
    build: ./frontend
    ports: ["3000:3000"]
    depends_on:
      - patient-service
      - appointment-service
      - doctor-service
    networks: [hospital-network]

networks:
  hospital-network:
    driver: bridge

```

14.3 Appendix B — Project Directory Structure

```
mikroservis/ ├── docker-compose.yml
├── activity-log-service/
│   ├── Dockerfile
│   ├── pom.xml
│   └── src/main/java/com/hospital/activitylog/
│       ├── ActivityLogApplication.java
│       └── ActivityLogVerticle.java
├── patient-service/
│   ├── Dockerfile
│   ├── pom.xml
│   └── src/main/java/com/hospital/patient/
│       ├── resource/PatientController.java
│       ├── resource/LogClient.java
│       └── model/Patient.java
├── appointment-service/
│   ├── Dockerfile
│   ├── pom.xml
│   └── src/main/java/com/hospital/appointment/
│       └── AppointmentService.java
├── doctor-service/
│   ├── Dockerfile
│   ├── pom.xml
│   └── src/main/java/com/hospital/doctor/
│       ├── DoctorApplication.java
│       ├── DoctorConfiguration.java
│       ├── model/Doctor.java
│       └── resource/DoctorResource.java
└── frontend/
    ├── Dockerfile
    └── package.json
```



```
└─ src/
   └─ App.jsx
   └─ index.css
   └─ components/
      └─ Dashboard.jsx
      └─ Patients.jsx
      └─ Appointments.jsx
      └─ Doctors.jsx
      └─ ActivityLogs.jsx
```